

SafeClub – Trésorerie sécurisée d'un club étudiant sur Ethereum

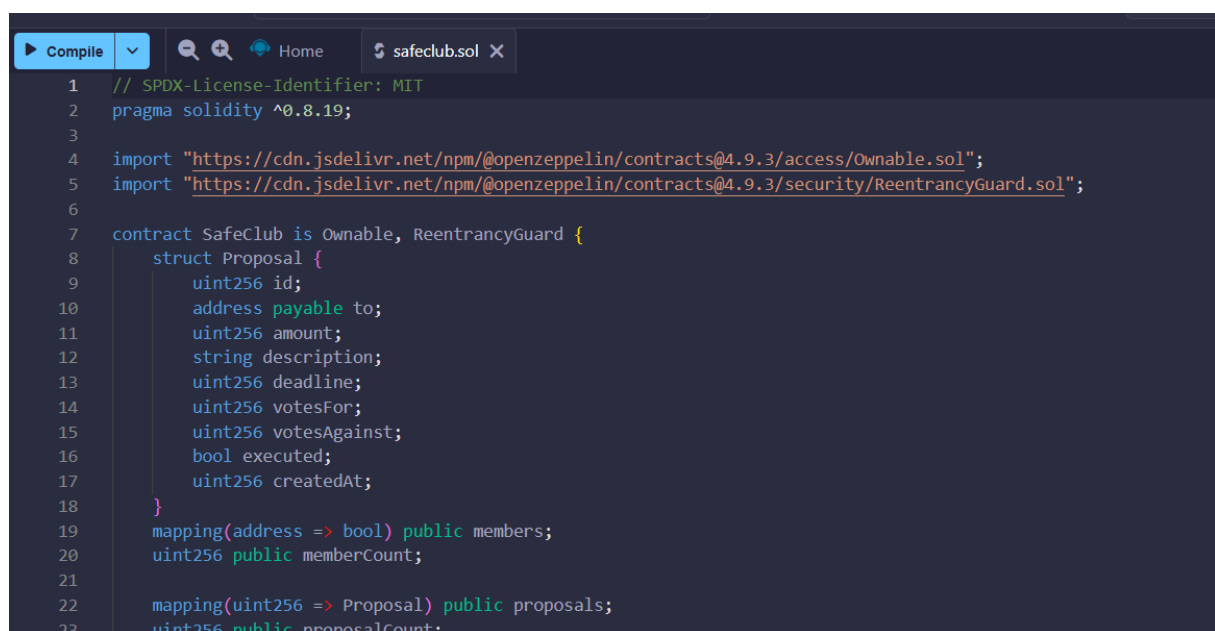
1. But du projet

SafeClub est un contrat intelligent (smart contract) qui permet à un club étudiant de gérer sa **trésorerie en ETH** (de l'argent sur la blockchain Ethereum), de façon transparente et sécurisée.

Fonctionnalités principales :

- **Gestion des membres** : le club peut ajouter ou retirer des membres qui ont le droit de participer.
- **Réception d'ETH** : le contrat peut recevoir des fonds (argent) en ETH, comme une "cagnotte".
- **Création de propositions de dépenses** : les membres peuvent proposer des dépenses avec un montant, un bénéficiaire, une description, et un délai pour voter.
- **Vote des membres** : chaque membre peut voter pour ou contre chaque proposition (1 seul vote par membre par proposition).
- **Exécution sécurisée** : si une proposition est acceptée (majorité + quorum), l'argent est transféré au bénéficiaire, mais seulement si toutes les règles sont respectées (vérifications de sécurité).

Explication code :



```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.19;
3
4 import "https://cdn.jsdelivr.net/npm/@openzeppelin/contracts@4.9.3/access/Ownable.sol";
5 import "https://cdn.jsdelivr.net/npm/@openzeppelin/contracts@4.9.3/security/ReentrancyGuard.sol";
6
7 contract SafeClub is Ownable, ReentrancyGuard {
8     struct Proposal {
9         uint256 id;
10        address payable to;
11        uint256 amount;
12        string description;
13        uint256 deadline;
14        uint256 votesFor;
15        uint256 votesAgainst;
16        bool executed;
17        uint256 createdAt;
18    }
19    mapping(address => bool) public members;
20    uint256 public memberCount;
21
22    mapping(uint256 => Proposal) public proposals;
23    uint256 public proposalCount;
```

```
24
25 mapping(uint256 => mapping(address => bool)) public hasVoted;
26
27 uint8 public quorumPercent = 50;
28
29 event MemberAdded(address indexed member);
30 event MemberRemoved(address indexed member);
31 event Deposit(address indexed from, uint256 amount);
32 event ProposalCreated(uint256 id, address indexed proposer, address to, uint256 amount, uint256 deadline);
33 event Voted(uint256 indexed id, address indexed voter, bool support);
34 event ProposalExecuted(uint256 id, address indexed executor);
35
36 modifier onlyMember() {
37     require(members[msg.sender], "Only members");
38     _;
39 }
40
41 constructor() {  infinite gas 1936400 gas
42     members[msg.sender] = true;
43     memberCount = 1;
44     emit MemberAdded(msg.sender);
45 }
```

```
46
47 receive() external payable {  undefined gas
48     emit Deposit(msg.sender, msg.value);
49 }
50
51 function deposit() external payable {  1733 gas
52     emit Deposit(msg.sender, msg.value);
53 }
54
55 function addMember(address _member) external onlyOwner {  infinite gas
56     require(_member != address(0), "zero address");
57     require(!members[_member], "already member");
58     members[_member] = true;
59     memberCount += 1;
60     emit MemberAdded(_member);
61 }
62
63 function removeMember(address _member) external onlyOwner {  infinite gas
64     require(members[_member], "not a member");
65     members[_member] = false;
66     memberCount -= 1;
67     emit MemberRemoved(_member);
68 }
```

```

69     function createProposal(
70         address payable _to,
71         uint256 _amountWei,
72         string calldata _description,
73         uint256 _durationSeconds
74     ) external onlyMember returns (uint256) {
75         require(_to != address(0), "invalid recipient");
76         require(_amountWei > 0, "amount > 0");
77         require(_durationSeconds >= 60, "duration too short");
78
79         proposalCount += 1;
80         uint256 id = proposalCount;
81
82         proposals[id] = Proposal({
83             id: id,
84             to: _to,
85             amount: _amountWei,
86             description: _description,
87             deadline: block.timestamp + _durationSeconds,
88             votesFor: 0,
89             votesAgainst: 0,
90             executed: false,
91             createdAt: block.timestamp

```

```

91             createdAt: block.timestamp
92         });
93
94         emit ProposalCreated(id, msg.sender, _to, _amountWei, proposals[id].deadline);
95         return id;
96     }
97
98     function vote(uint256 _proposalId, bool _support) external onlyMember {
99         Proposal storage p = proposals[_proposalId];
100         require(p.id != 0, "proposal not found");
101         require(block.timestamp <= p.deadline, "voting closed");
102         require(!hasVoted[_proposalId][msg.sender], "already voted");
103         require(!p.executed, "already executed");
104
105         hasVoted[_proposalId][msg.sender] = true;
106
107         if (_support) {
108             p.votesFor += 1;
109         } else {
110             p.votesAgainst += 1;
111         }
112
113         emit Voted(_proposalId, msg.sender, _support);

```

```
113     emit Voted(_proposalId, msg.sender, _support);
114 }
115
116 function isAccepted(uint256 _proposalId) public view returns (bool) {  infinite gas
117     Proposal storage p = proposals[_proposalId];
118     uint256 totalVotes = p.votesFor + p.votesAgainst;
119
120     uint256 minVotesRequired = 1;
121
122     if (totalVotes < minVotesRequired) return false;
123
124     return p.votesFor > p.votesAgainst;
125 }
126
127 function executeProposal(uint256 _proposalId) external  infinite gas
128     Proposal storage p = proposals[_proposalId];
129     require(block.timestamp > p.deadline, "voting not finished");
130     require(!p.executed, "already executed");
131     require(isAccepted(_proposalId), "proposal not accepted");
132     require(address(this).balance >= p.amount, "insufficient balance");
133
134     p.executed = true;
```

```
131     require(isAccepted(_proposalId), "proposal not accepted");
132     require(address(this).balance >= p.amount, "insufficient balance");
133
134     p.executed = true;
135
136     (bool sent, ) = p.to.call{value: p.amount}("");
137     require(sent, "transfer failed");
138
139     emit ProposalExecuted(_proposalId, msg.sender);
140 }
141
142 function getContractBalance() external view returns (uint256) {  423 gas
143     return address(this).balance;
144 }
145 }
146
```

a) Import OpenZeppelin

import

"https://cdn.jsdelivr.net/npm/@openzeppelin/contracts@4.9.3/access/Ownable.sol";

import

"https://cdn.jsdelivr.net/npm/@openzeppelin/contracts@4.9.3/security/ReentrancyGuard.sol";

-**Ownable** : fournit une fonctionnalité de propriétaire unique (owner) du contrat qui peut faire certaines opérations exclusives

- **ReentrancyGuard** : protège contre les attaques de type "reentrancy" où un attaquant pourrait appeler plusieurs fois une fonction de transfert d'argent pour voler des fonds.

b) Structure Proposal

```
struct Proposal {  
    uint256 id;  
    address payable to;  
    uint256 amount;  
    string description;  
    uint256 deadline;  
    uint256 votesFor;  
    uint256 votesAgainst;  
    bool executed;  
    uint256 createdAt;  
}
```

c) Gestion des membres

mapping(address => bool) public members;
uint256 public memberCount;

- members : mapping (adresse → bool) indiquant qui est membre (true) ou non (false)
- memberCount : nombre total de membres actifs

Le **propriétaire** (owner) peut ajouter ou retirer des membres avec ces fonctions :

```
function addMember(address _member) external onlyOwner { ... }
```

```
function removeMember(address _member) external onlyOwner { ... }
```

d) Réception d'ETH

```
receive() external payable { ... }
```

```
function deposit() external payable { ... }
```

- Le contrat peut recevoir des ETH via la fonction spéciale receive() (transfert direct) ou via la fonction deposit() explicite.

-Un événement est émis pour enregistrer chaque dépôt.

e) Création des propositions

```
function createProposal(  
    address payable _to,  
    uint256 _amountWei,  
    string calldata _description,  
    uint256 _durationSeconds  
) external onlyMember returns (uint256) { ... }
```

- Un membre peut proposer une dépense : montant, destinataire, description et durée de vote (minimum 60 secondes).
- La proposition est enregistrée, un ID est attribué, et un événement est émis.

f) Vote des membres

```
function vote(uint256 _proposalId, bool _support) external onlyMember { ... }
```

- Un membre peut voter **une seule fois** par proposition, pour ou contre.
- Le vote est enregistré et un événement est émis.
- Le vote n'est possible que si la date limite n'est pas dépassée et que la proposition n'est pas déjà exécutée.

g) Vérification d'acceptation

```
function isAccepted(uint256 _proposalId) public view returns (bool) { ... }
```

- Pour qu'une proposition soit acceptée :
 - Il faut atteindre un **quorum** (exemple : au moins 50% des membres ont voté)
 - Il faut que les votes pour soient plus nombreux que les votes contre

h) Exécution sécurisée

```
function executeProposal(uint256 _proposalId) external nonReentrant onlyMember { ... }
```

Un membre peut exécuter une proposition uniquement si :

- La période de vote est terminée
- La proposition est acceptée
- Elle n'a pas encore été exécutée
- Le contrat a assez d'ETH pour payer

La fonction est protégée contre la reentrancy (nonReentrant)

Le montant est envoyé au bénéficiaire

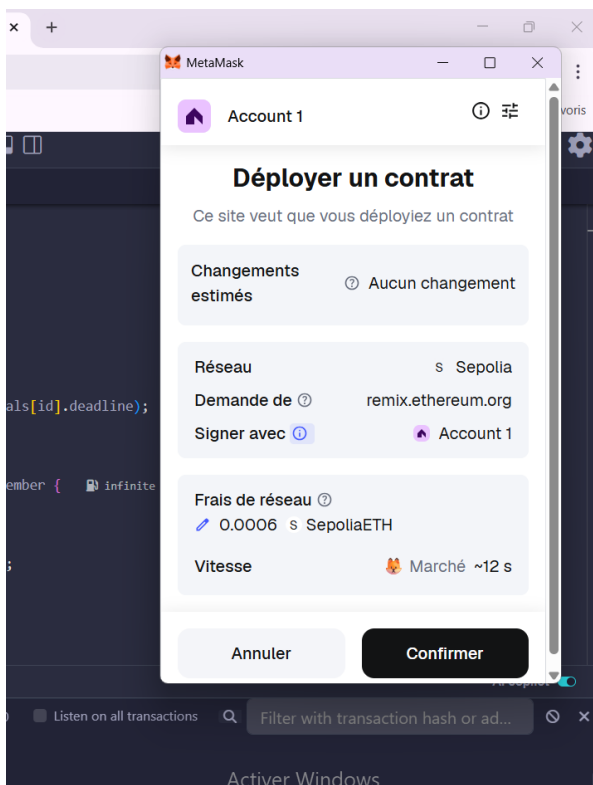
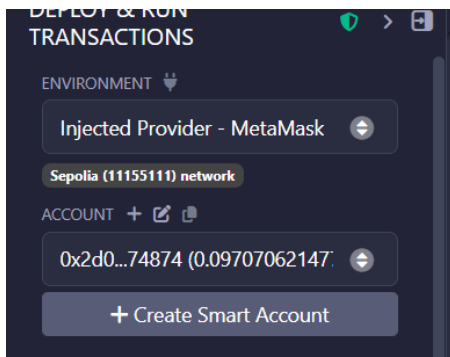
Un événement est émis

i) Fonction utilitaire

```
function getContractBalance() external view returns (uint256) {  
    return address(this).balance;  
}
```

Permet de voir combien d'ETH est stocké dans le contrat.

Deployment



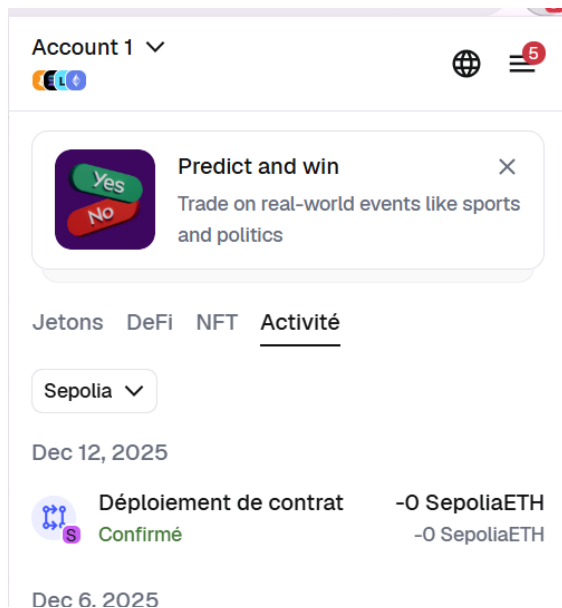
Etherscan spécifique au réseau de test Sepolia

[This is a Sepolia Testnet transaction only]	
Transaction Hash:	0x11b110e227198f8593cd66a6b2e842d3016240c348858fd46878d4780e3aea93
Status:	Success
Block:	9827019 1 Block Confirmation
Timestamp:	13 secs ago (Dec-12-2025 09:31:12 PM UTC)
From:	0x2D0a1DDA44C5850A3d181FFf333f4D5b42b74874
To:	[0x86ecdbe2e2766567d232ef73b0ff1bef088a8983 Created]
Value:	0 ETH
Transaction Fee:	0.003416532018221504 ETH
Gas Price:	1.500000008 Gwei (0.000000001500000008 ETH)
More Details: + Click to show more	

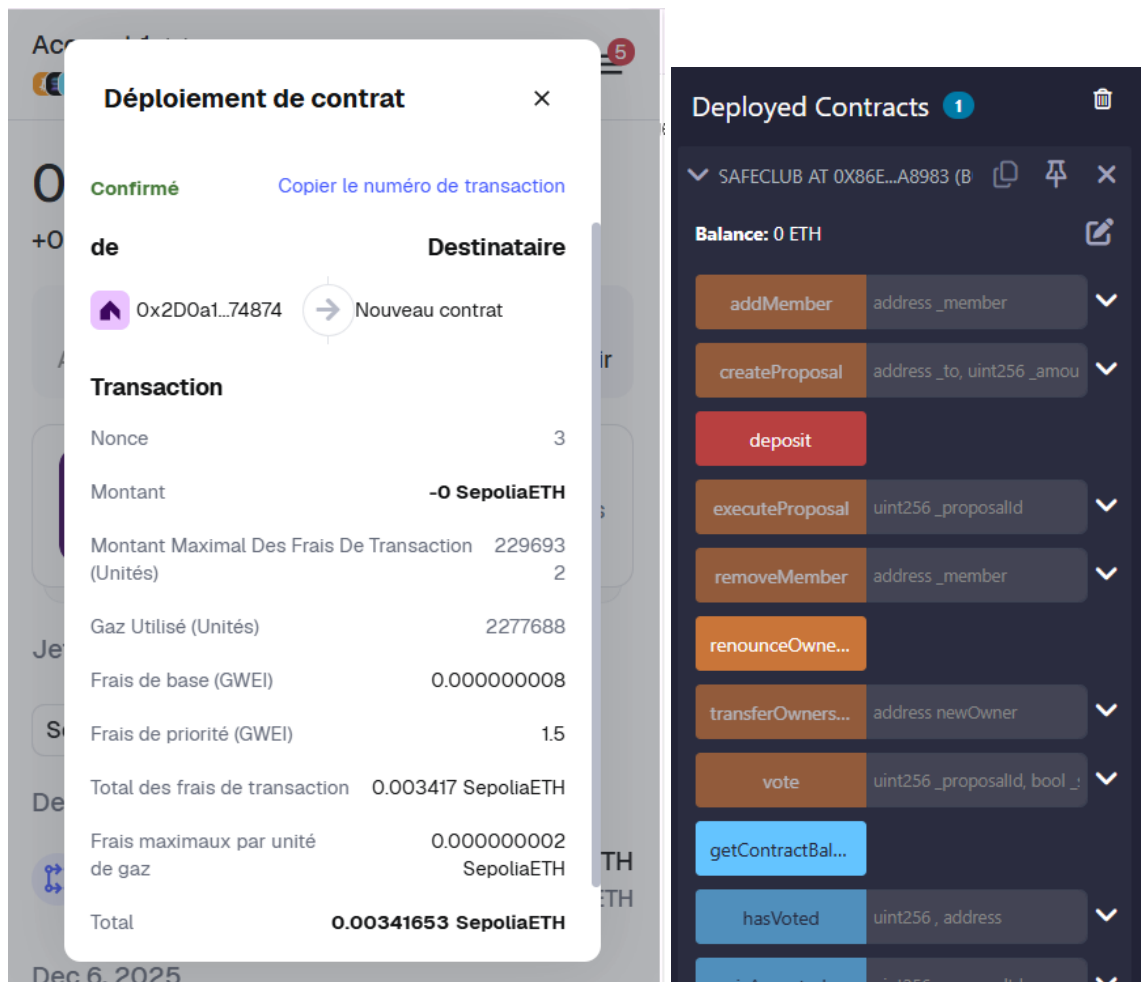
Détail de ta transaction sur cette page :

- Transaction Hash :**
 C'est un identifiant unique qui représente ta transaction sur la blockchain.
 Ici :
0x11b110e227198f8593cd66a6b2e842d3016240c348858fd46878d4780e3aea93
- Status: Success :**
 La transaction a été validée avec succès, donc ton contrat a bien été déployé.
- Block : 9827019 :**
 La transaction est incluse dans ce bloc de la blockchain.
- Timestamp :**
 L'heure précise à laquelle la transaction a été validée (ici, il y a 13 secondes).
- From :**
 L'adresse Ethereum qui a envoyé la transaction (ton compte MetaMask, celui qui a déployé le contrat).
- To :**
 L'adresse du contrat déployé (le résultat de ta transaction).
 C'est l'adresse où ton smart contract SafeClub est désormais disponible sur Sepolia.
- Value :**
 Montant envoyé avec la transaction, ici 0 ETH car tu déployais juste un contrat.
- Transaction Fee :**
 Frais de transaction payés (en ETH) pour faire cette opération.
- Gas Price :**
 Le prix du gas au moment de la transaction (combien tu payes par unité de gas).

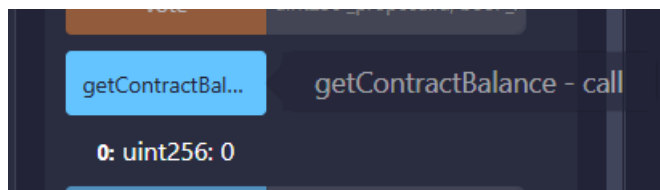
l'adresse du contrat : 0xdc005b33Fe8b5608E6Bf23498Bc98447997ECa9A



=>contrat bien déployé



1.

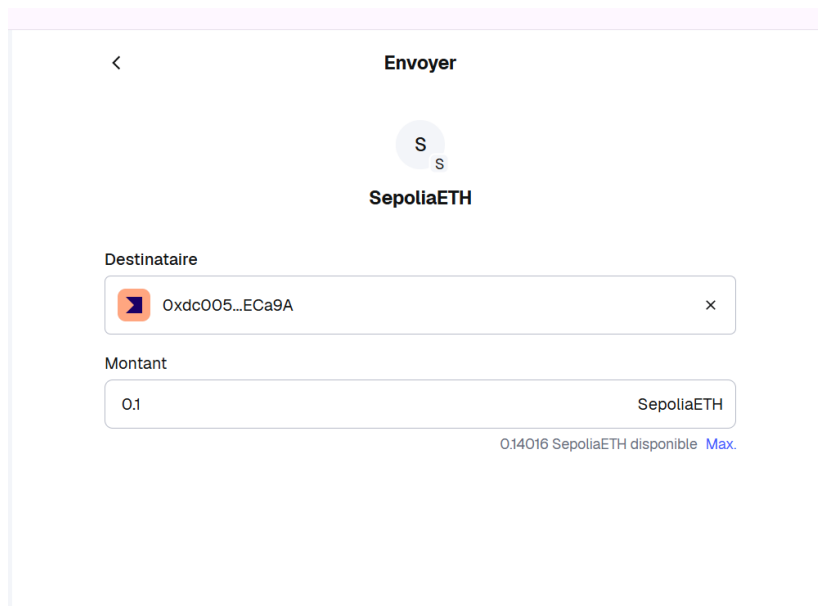


n'as pas encore envoyé d'ETH

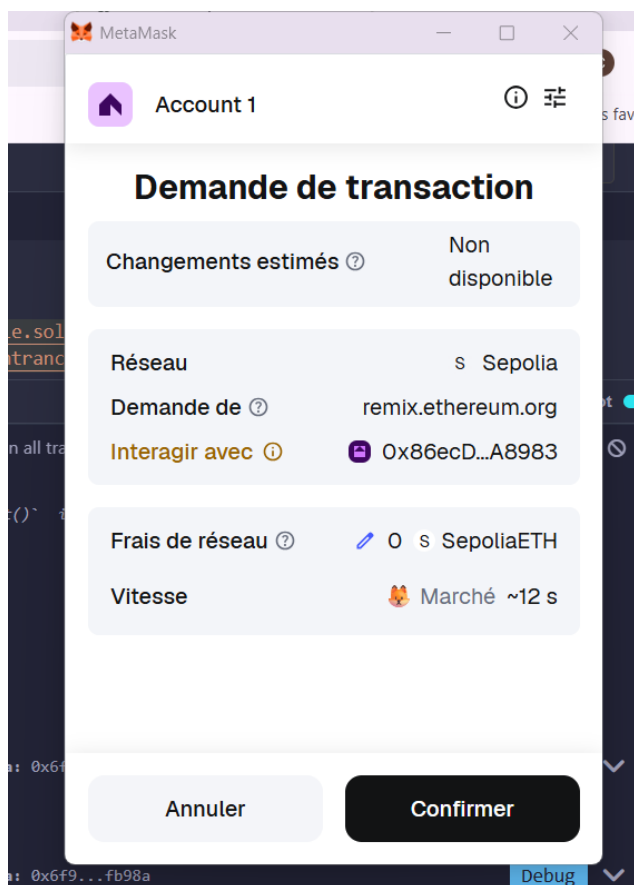
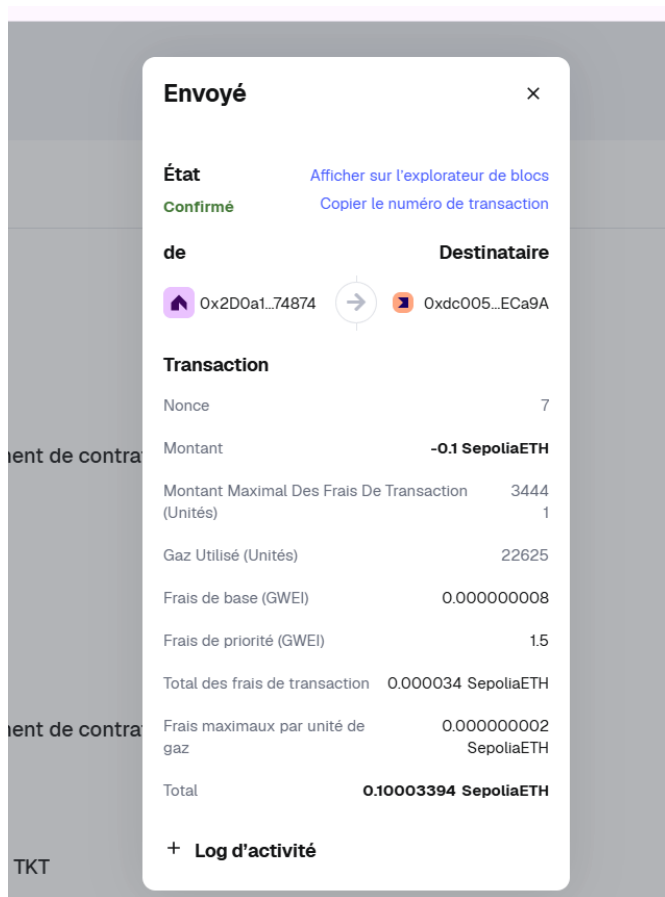


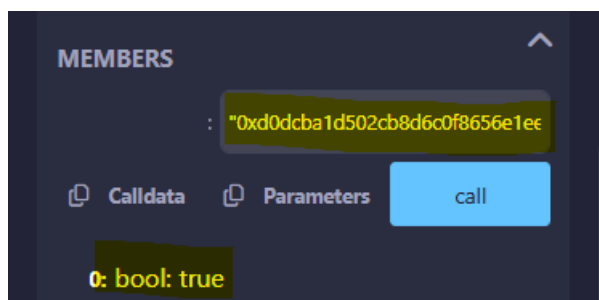
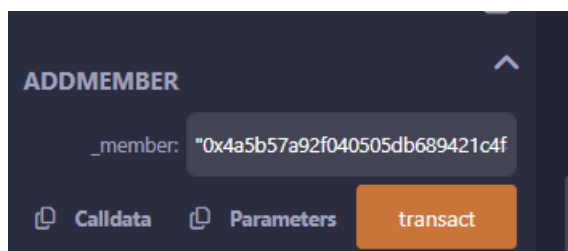
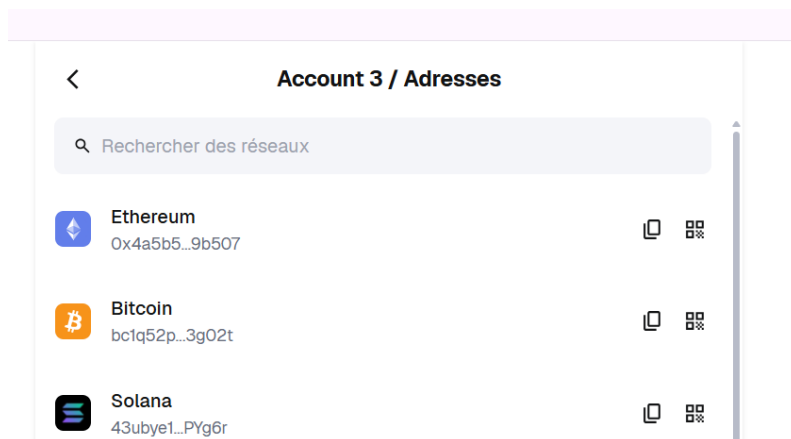
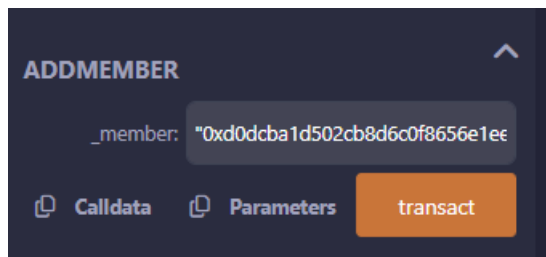
2.

Envoyer de l'ETH au contrat (alimenter la trésorerie)

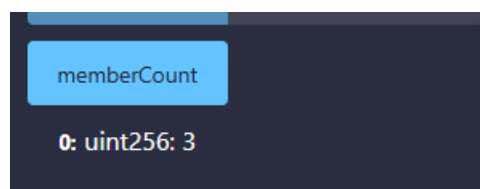


=> envoie une petite somme (exemple 0.1 ETH) depuis metamask a l adresse de contrat





⇒ Les membres ont été bien ajouté members return true



⇒ 2membres

4. créer une proposition de dépense

bénéficiaire / Adresses

🔍 Rechercher des réseaux



Ethereum
0x8d39d...5b17a



Bitcoin
bc1qOs8...khcnu



Solana
D1AdA3K...XE13X



CREATEPROPOSAL

```
_to: 0x8d39ddceedca1af7416a6e00553a
```

```
_amountWei: 500000000000000000
```

```
_description: "Achat matériel"
```

```
_durationSeconds: 120
```



Calldata



Parameters

transact

la période de vote= 2mins

```
0 Listen on all transactions  Filter with transaction

decoded input
{
  "address_to": "0x8d39ddCeEDca1Af7416A6e00553A37Bd52D5B17a",
  "uint256_amountWei": "5000000000000000",
  "string_description": "Achat1",
  "uint256_durationSeconds": "160"
}

decoded output
-

logs
[
  {
    "from": "0xdc005b33Fe8b5608E68f234988c98447997ECa9A",
    "topic": "0x0498efc40851dd66b079b44ac06f0291de83b764e409f0e2727693d9a48a39f8",
    "event": "ProposalCreated",
    "args": {
      "0": "2",
      "1": "0x2D0a1DDA44C5850A3d181fFF33f4D5b42b74874",
      "2": "0x8d39ddCeEDca1Af7416A6e00553A37Bd52D5B17a",
      "3": "5000000000000000",
      "4": "1765579804"
    }
  }
]
```

proposalCount

0: uint256: 2

PROPOSALS

: 1

Calldata Parameters call

0: uint256: id 1

1: address: to 0x8d39ddCeEDca1Af7416A6e00553A37Bd52D5B17a

2: uint256: amount 5000000000000000000

3: string: description Achat matériel

4: uint256: deadline 1765579056

5: uint256: votesFor 0

6: uint256: votesAgainst 0

7: bool: executed false

8: uint256: createdAt 1765578936

PROPOSALS

: 2

Calldata Parameters call

0: uint256: id 2

1: address: to 0x8d39ddCeEDca1Af7416A6e00553A37Bd52D5B17a

2: uint256: amount 5000000000000000000

3: string: description Achat1

4: uint256: deadline 1765579804

5: uint256: votesFor 1

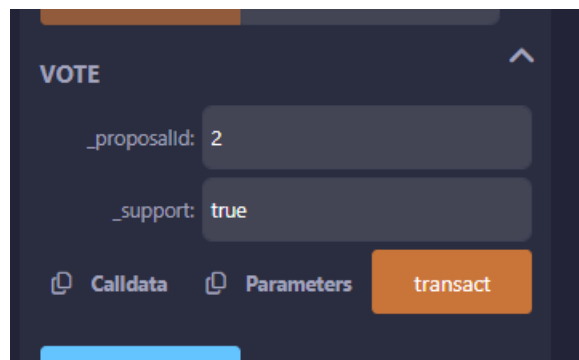
6: uint256: votesAgainst 0

7: bool: executed false

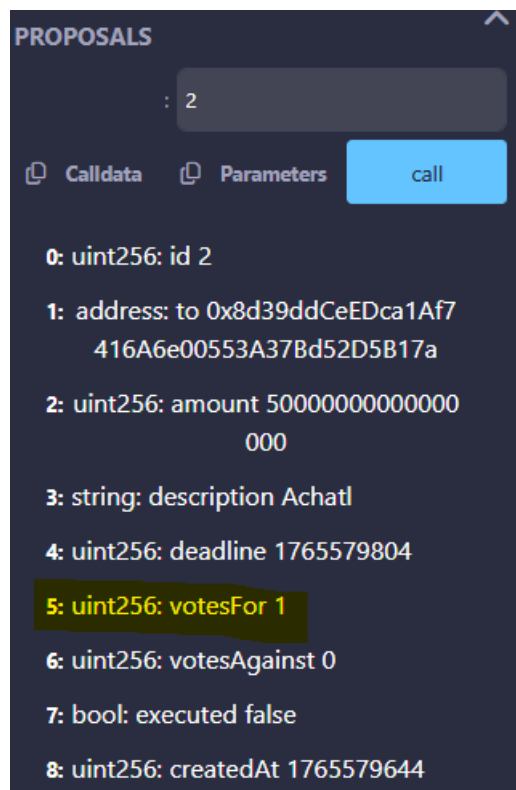
8: uint256: createdAt 1765579644

5. Le vote

Changer le compte dans metamask a chaque fois pour voter



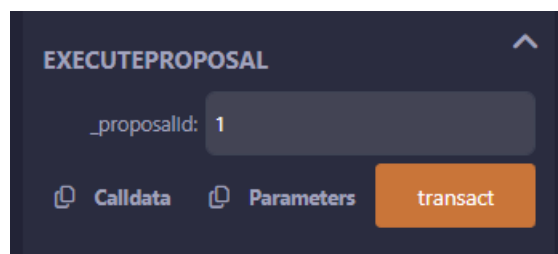
The screenshot shows a dark-themed interface with the title "VOTE" at the top left. Below the title, there are two input fields: the first is labeled "_proposalId:" and contains the value "2"; the second is labeled "_support:" and contains the value "true". At the bottom of the form, there are three buttons: "Calldata" and "Parameters" are light blue with document icons, and "transact" is an orange button.



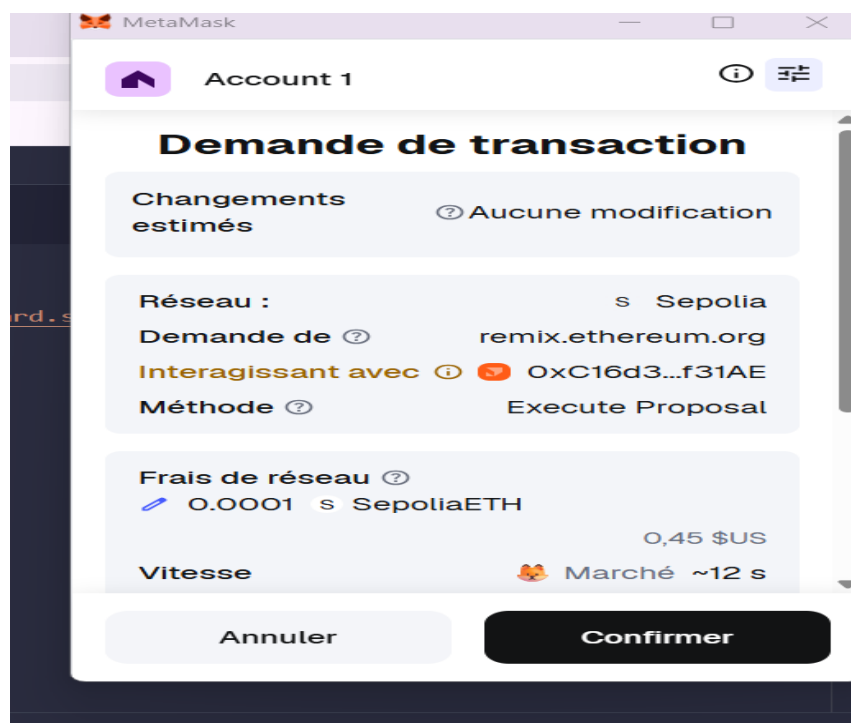
The screenshot shows a dark-themed interface with the title "PROPOSALS" at the top left. Below the title, there is a search bar containing the value "2". Below the search bar, there are three buttons: "Calldata" and "Parameters" are light blue with document icons, and "call" is a blue button. Below the buttons, there is a list of parameters for the proposal:

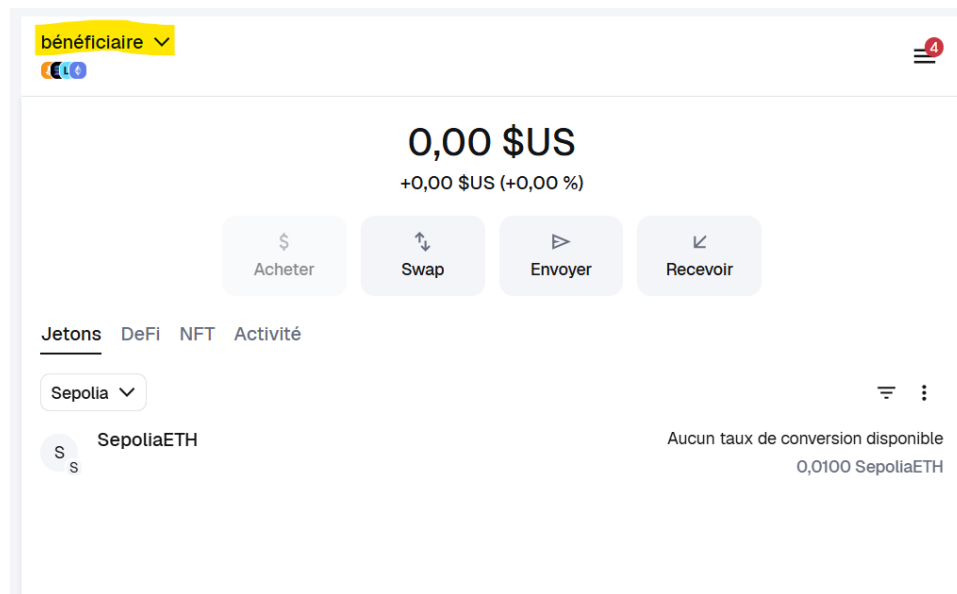
- 0: uint256: id 2
- 1: address: to 0x8d39ddCeEDca1Af7416A6e00553A37Bd52D5B17a
- 2: uint256: amount 5000000000000000000
- 3: string: description Achat1
- 4: uint256: deadline 1765579804
- 5: uint256: votesFor 1
- 6: uint256: votesAgainst 0
- 7: bool: executed false
- 8: uint256: createdAt 1765579644

6.Execution de Proposal

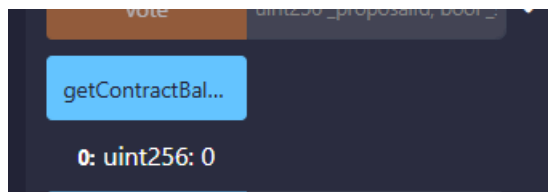


The screenshot shows a dark-themed interface with the title "EXECUTEPROPOSAL" at the top left. Below the title, there is an input field labeled "_proposalId:" containing the value "1". At the bottom of the form, there are three buttons: "Calldata" and "Parameters" are light blue with document icons, and "transact" is an orange button.





=>l'adresse bénéficiaire a bien **reçu l'ETH**



=>le solde du contrat **diminue (devient 0)**

Modèle de menaces

Scénario 1 : Attaque par réentrance (Reentrancy Attack)

Un attaquant pourrait exploiter une fonction effectuant un appel externe (call) avant de mettre à jour l'état interne, provoquant un double retrait ou une manipulation des fonds.

Impact : Perte de fonds du contrat.

Contre-mesure : Utilisation d'un modificateur nonReentrant empêchant les appels réentrants.

Scénario 2 : Manipulation du temps (Timestamp Manipulation)

Les mineurs peuvent ajuster légèrement block.timestamp, influençant les délais de vote pour des propositions.

Impact : Un mineur pourrait précipiter ou retarder la clôture des votes.

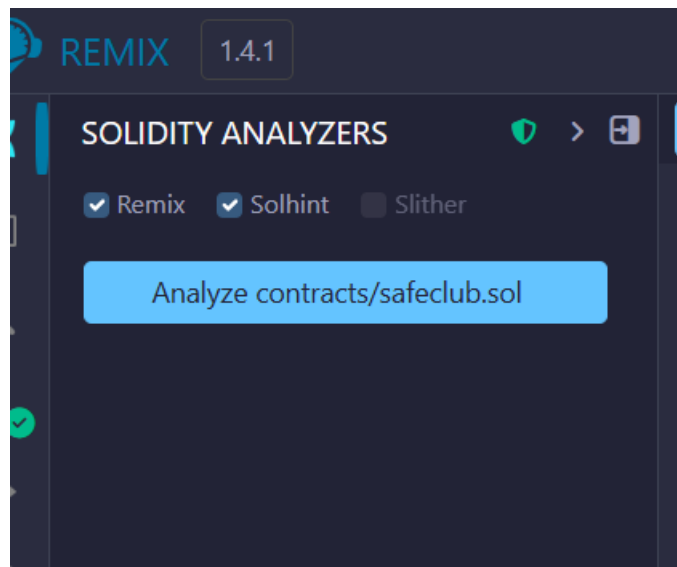
Contre-mesure : Le timestamp est uniquement utilisé pour gérer des délais peu critiques et ne sert pas à des calculs financiers sensibles.

Scénario 3 : Votes multiples ou non autorisés

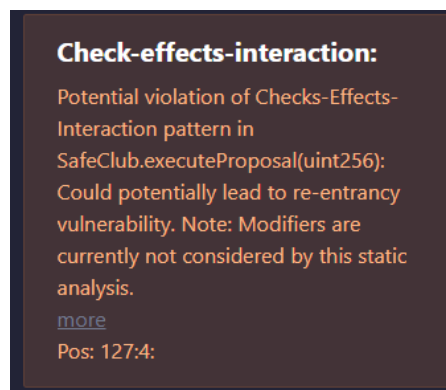
=> Pas de double exécution

Analyse statique (détection des failles de sécurité, mauvaises pratiques ou risques logiques)

Outils : Remix Analyzer



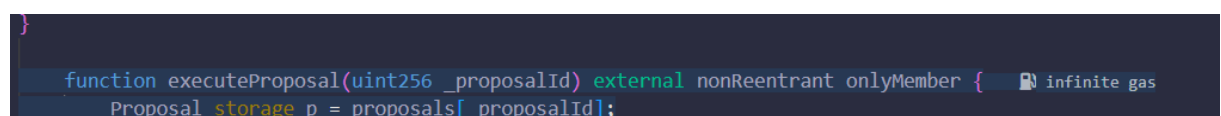
a-



La fonction executeProposal effectue un appel externe (p.to.call{value: p.amount}("")).

Les appels externes peuvent ouvrir la porte à des attaques de **reentrancy**.

=>Le risque est correctement mitigé grâce à nonReentrant



b-

Block timestamp:

Use of "block.timestamp":

"block.timestamp" can be influenced by miners to a certain degree. That means that a miner can "choose" the block.timestamp, to a certain degree, to change the outcome of a transaction in the mined block.

[more](#)

Pos: 87:22:

block.timestamp can be influenced by miners

=>Le block.timestamp représente la date et l'heure du bloc miné, mais il peut être légèrement modifié par les mineurs. Ici, il sert seulement à gérer les délais de vote, ce qui est acceptable. Il ne doit pas être utilisé pour des calculs critiques ou sensibles.

c-

Low level calls:

Use of "call": should be avoided whenever possible. It can lead to unexpected behavior if return value is not handled properly. Please use Direct Calls via specifying the called contract's interface.

[more](#)

Pos: 136:24:

En Solidity, il existe plusieurs manières d'envoyer de l'Ether à une adresse : transfer, send et call. Parmi ces méthodes, call est la plus bas niveau et la plus flexible, mais aussi la plus risquée si elle est mal utilisée.

- **Pourquoi call est risqué ?**

Parce qu'il exécute un appel externe direct, ce qui peut entraîner des failles de sécurité (comme la réentrance) si on ne contrôle pas bien ce qui se passe après.

=>'utilisation de la fonction low-level call dans le contrat peut présenter des risques de sécurité, notamment des attaques par réentrance. Cependant, dans ce cas, le retour de call est systématiquement vérifié et un modificateur nonReentrant est utilisé pour sécuriser la fonction.

d-

Gas costs:

Gas requirement of function
SafeClub.addMember is infinite: If the
gas requirement of a function is
higher than the block gas limit, it
cannot be executed. Please avoid
loops in your functions or actions that
modify large areas of storage (this
includes clearing or copying arrays in
storage)
Pos: 55:4:

Gas requirement of function is infinite

Analyse :

- Aucune boucle
- Aucune itération sur tableaux
- Pas de nettoyage massif du storage

Interface Web

Interface Utilisateur

- Le site affiche clairement le solde total d'Ether détenu par le contrat.
- Il liste toutes les propositions avec les détails suivants :
 - ID de la proposition
 - Description
 - Destinataire de la transaction
 - Montant en Ether
 - Date de création et deadline
 - Nombre de votes pour et contre
 - Statut d'exécution (oui/non)

Technologies utilisées

- **Ethers.js** pour interagir avec le contrat depuis le navigateur.
- **HTML/CSS/JavaScript** pour la construction de l'interface utilisateur.

=>Le site web permet une interaction facile avec la blockchain

